务　　过　　检查　　记唤　　进　　复

检查　　还没登记 这个窗口里到达时，等待方错过了这次通知，之后就没有人再来唤醒它了。本章大多数改动都对准这个竞争窗口，以及由此派生出的几个具体问题：超时后旧等待项还留在队列里，子进程状态变化和父进程登记　　间

队　间　　样　竞

调度框架，改动集中在进入阻塞和重新唤醒之间的那段路径。任务在就绪、运行和阻塞之间循环，执行完毕后退出调度流程，整体关系如图所示。

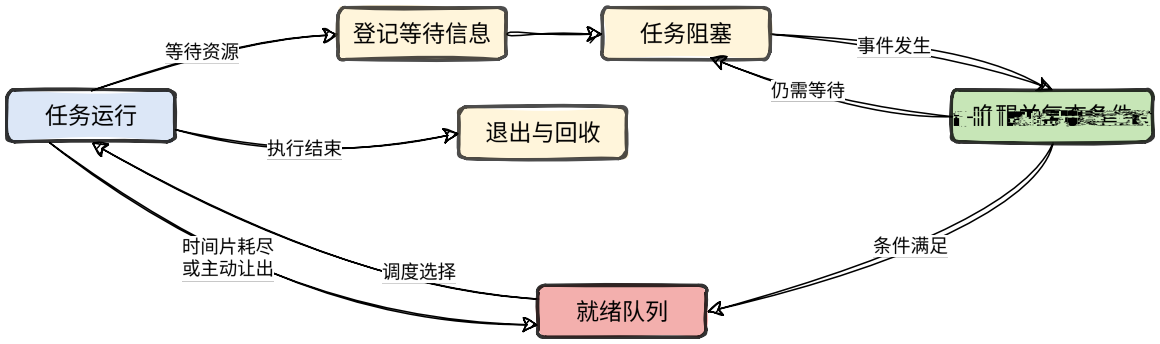


图　　调度、阻塞与唤醒的总体循环

3.1

经　　运　　务调　　axtask　　务　态　运　队　axsched

调度器按时间片轮转，多核下每个　　绪队　　码唤

队　　实现　时　　进　　这

础　进　补

问题集中在事件交错时的状态处理上。　　项　　码

态　时　　记录还　队　　说　　达

项　　续唤　额　FUTEX_WAIT_BITSET　绝对　时间　对时长　时

设　　样　　队　　进　态变　　进　记　　间　　样

调度框架，改动集中在三个环节：等待者如何登记，通知如何与阻塞状态配合，等待结束后如何清理记录。对应关系如下：

	线	
务调	务 态 时间片和 亲和性	调 亲 图
唤	队	调整 唤醒过程，补充条件复查和等待项清理
时处	础 进 时	对 绝对时间 过滤旧事件
	码 队	态 时语义
进	础	态变 记 复查

调度与同步机制的主要增量

这些修改共同作用于任务离开 获 过 难处 态转换 时发 务检查 记 时，事件可能已经到达；任务准备进入阻塞时，通知也可能先一步发生。下面的叙述会先用任务状态、抢占和上下文切换说明调度基础，再围绕这一竞争窗口讨论通用阻塞唤醒机制，最后分析定时等待、子进程等待和 处

3.2

3.2.1

务调 围绕 种 态 开 调 务 哪 种资 务 种 哪种 态

```
pub enum TaskState {
    Running = 1, //      CPU
    Ready   = 2, //
    Blocked = 3, //
    Exited  = 4, //
}

pub struct TaskInner {
    //
    state: AtomicU8,
    // ...
}
```

码 务 态 结构

state 务 态 时 进 维护 业务 获 须让 务 Blocked Ready 调 处 这 态转换 经满 种 态 间 转换关 图

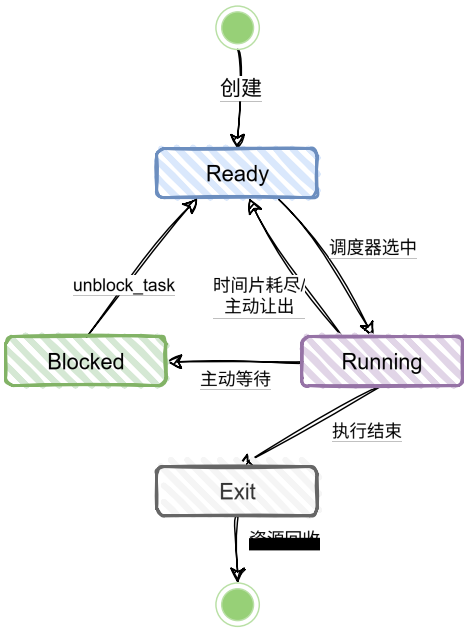


图　　态转换关

　　务　　Ready　态进　　绪队　　调　　选　　变为 Running　　动让
务　　备运　　Ready　　资　　务则进　Blocked　　unblock_task　复
为 Ready　　务执　结　　进　Exited　　资　　开　态　　时
　进　　过这组　态转换　　调
　　态　　过　　态　运　队　　唤　检查　　时　达时　唤
应该　　许　处　Blocked　　务　　进　　绪队

3.2.2

　　务　　轮　　环　　绪队
处　Ready　态　　务　调　　队头选择　　务　　务　　认时间　　时钟　　时
间　　务　　队　　转交给后面的任务。

　　时钟中断每到来一次，scheduler_timer_tick　调　　时钟节　处
务　时间　　计　归　时　　设　　抢　标　　处　　锁　　换　务
　码　开　　抢　　临　　执　　调

时钟中断　时间片减一　设置待抢占标志　开临　调

　　时间片归零后，内核先记录抢占请求，等执行离开禁止抢占的临界区再调度。任务如果在修改运行队列、处理中断或持有关键锁时被换出，其他　　间　态　迟
抢　　换　　态　　运　队　锁　处　　结
　　务　运　　进　　务　　务时运　　闲　务
　　务

3.2.3

　　两节　经说　　务　态　变　　时间　　发　抢　　节　　调
　开　　继续　　务　态处　毕　resched　　绪队　选
　务　　队　为　　让　　务暂时

```rust
fn resched(&mut self) {
    //                                idle
    let next = self.scheduler.lock().pick_next_task()
        .unwrap_or_else(|| unsafe {
            IDLE_TASK.current_ref_raw().get_unchecked().clone()
        });

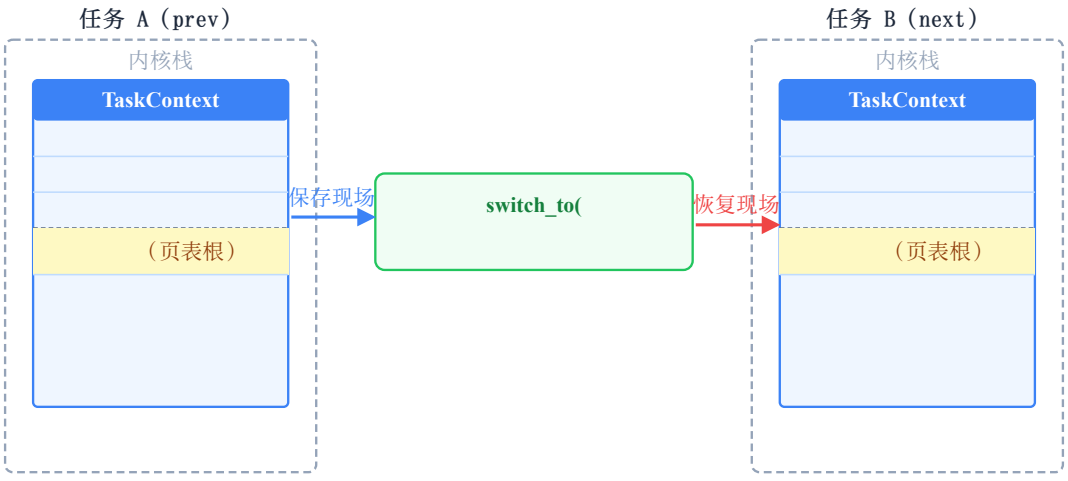
    self.switch_to(crate::current(), next);
}

fn switch_to(&mut self, prev: CurrentTask, next: AxTaskRef) {
    //
    next.set_state(TaskState::Running);

    //                  复
    let prev_ctx = prev.ctx_mut_ptr();
    let next_ctx = next.ctx_mut_ptr();
    unsafe { (*prev_ctx).switch_to(&*next_ctx) };
}
```

码　　选择　务　　换现场

选出任务后，switch_to　　标记为 Running　　换　务现场　这　　现场　　务暂　运　　务　　TaskContext　　栈　针　　换时　　这　值　　务　　务　TaskContext　复　务　　选　时　　运　　图　　两　务　TaskContext　别　　栈　switch_to　值　　读入；其中 satp 让切换完成的瞬间就生效了新的地址空间。



satp 在切换同一时刻更新，任务 B 运行后立即使用自己进程的页表

图　　现场传递

对于用户任务，　　创　　务时　该进　　页　　TaskContext　证　换　确　　间　户态　　则　　UserContext　户　暂　时　　记录　　抢　请　　换　　连　　这

设置待抢占标志　　达　　调　　选择下一任务　　现场　　复　现场

3.2.4 CPU

　　务　　亲和性掩码 cpumask　　哪　　运
码　　时，任务就不会进入其他　　绪队
　　务　认　　经启　　务创　　唤　时　调　　cpumask　许
围　轮转选择　　务　该　　绪队

```
fn select_run_queue_index(cpumask: AxCpuMask) -> usize {
    loop {
        //       CPU
        let index = RUN_QUEUE_INDEX.fetch_add(1, Ordering::SeqCst)
            % axconfig::plat::CPU_NUM;
        //                    CPU
        if cpumask.get(index) {
            return index;
        }
    }
}
```

　　码　　亲　　围　选择
　这段逻辑按　　编号轮转，并跳过亲和性掩码不允许的　　运　队　设计　　这　　实际　　户　图　　设　　务迁
　　户传　　亲　　围　　张　图　　对应　　这张　图　　过
　　实际　　读　　为　　务　码　　户
　过　　sched_getaffinity 查询范围，通过 sched_setaffinity　　围

```
//           每           CPU
for cpu_id in 0..axconfig::plat::CPU_NUM {
    if user_mask[cpu_id / 8] & (1 << (cpu_id % 8)) != 0 {
        cpu_mask.set(cpu_id, true);
    }
}

//
let curr = current().clone();
curr.set_cpumask(cpu_mask);

//     CPU                              CPU
if !cpu_mask.get(axhal::percpu::this_cpu_id
```

```
    current_run_queue().migrate_current(migration_task);
}
```

码 亲和性掩码的设置与任务迁移

码 还 检查 务 该 许 务 继续运
经 务迁 许 运 队 设 变 务
实际运 查询 值

户调 `sched_yield` 动让 务 绪队 调 选择
务运 这 务暂时 紧 让 绪 务 执

3.3

续 这 实际 处
唤 `block_on` 统 暂 复
务 为 单 实现 逻辑 确维护 态
检查 则竞 这 节 讨论

3.3.1

调度器只知道任务处于 `Blocked` 务 对 间还
协议 还 继续检查 记录
变 时 务
读 为 务 检查缓 读 则
记 暂时未完成 务 调
读 务继续检查

检查条件	检查
发 调用	执 检查

图 记

关键步骤是登记 检查 检查结 达
时 还 调 记 复查缓 读 务 经
达 为
还 检查 变 证资
务 读 务 时 时 经 务 务
继续

3.3.2 Waker PollSet

给 对 务唤 对 运 队
态发 变 时调 `wake()` 继续使用这一接口，把新增
的 务唤

PollSet 组 终 对 过 记
has_waiters 时 唤 时 锁 轮
锁 调 锁 护 继续 发调 wake()

```
pub struct PollSet {
    inner: Lazy<Mutex<Inner>>,
    //              wake
    has_waiters: AtomicBool,
}

pub fn wake(&self) -> usize {
    if !self.has_waiters.load(Ordering::Acquire) {
        return 0;
    }
    //                    Waker
    let (wakers, len) = self.inner.lock().take_wakers();
    self.has_waiters.store(false, Ordering::Release);
    //
    for waker in wakers.iter().take(len) {
        unsafe { waker.assume_init_read() }.wake();
    }
    len
}
```

码 唤
take_wakers 时 续 这 记录
负责 检查 业务对 满 检
查缓 进 检查 态 检查 标记

3.3.3 block_on

过 复轮询 统调 结 产 时暂
务 block_on 连接这两种执行方式: Ready 时，系统调用取得结果；返回 Pending
时，当前任务准备阻塞，把 给 务 务选择 现场 换 调

```
fn wake_by_ref(self: &Arc<Self>) {
    if let Some(task) = self.task.upgrade() {
        //
        *self.woke.lock() = true;
        //                  适
        select_run_queue(&task).unblock_task(task, false);
    }
}

match future.poll(&mut context) {
    //                                Blocked
    Poll::Pending if !*woke => run_queue.blocked_resched(woke),
```

```
    //
    Poll::Pending => yield_now(),
    Poll::Ready(output) => return output,
}
```

码 唤

Pending 达 发 邻时 woke 设为真，再尝试唤醒任务；阻塞路径则在改变任务状态前检查这个标记。通知先到时，任务不再进入 Blocked 让 轮询 时 unblock_task 务 绪队 woke 记录 轮轮询 经 补 态 务 态 间

图 对比了这两条路径的时序差异。

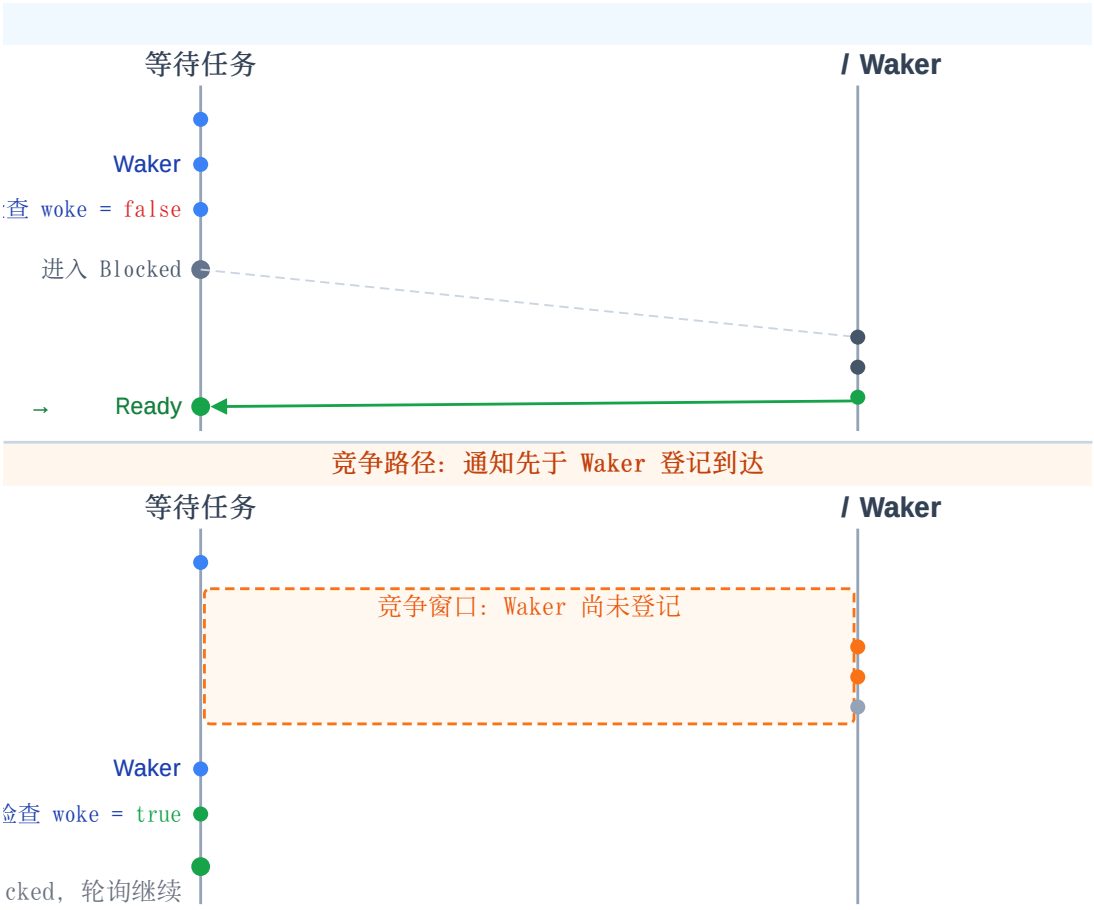


图 标记覆盖竞争路径的时序对比

结 interruptible 时检查 务 态 结 层 统调 EINTR 时间 规则 启

时 结 还 须删 轮 记 则 续 产 唤 扰唤 统计 经 则按等待项精确清理，具体过程将在 节说明。

3.4

节建立了 检查条件、登记 复查 节选择两
应 说 这 时 关 时间 达 进 关 现 报
态 业务 唤 终结 务 复 检
查

3.4.1

时 统调 选择时间 标统 换 为 时间 单调时钟
进 实时时钟对应 统 历时间 实际时
时间参数分为相对时长和绝对时间。相对时长需要加到当前时间上，绝对时间本身就是截止时间；如果截止时间已经到达，等待立即结束。统一转换后，后续逻辑只需比较 时间
达 时间 样 `FUTEX_WAIT_BITSET` 时语义
`nanosleep` 单调时钟 对 `clock_nanosleep` 还允许选择时钟，并通过 `TIMER_ABSTIME` 绝对时间 间 结 户
`EINTR` 时间

3.4.2

时 时间 发 时 设 队
仅 时间 `BTreeMap` 时
时间 `deadline` 递 `seq` 组成唯一的 `TimerKey` 时 务
时 态 发 应

```
struct TimerKey {
    //
    deadline: Duration,
    //
    seq: u64,
}

struct AlarmRuntime {
    //
    next_seq: u64,
    //
    timers: BTreeMap<TimerKey, Entry>,
}
```

码 时 队 关键 结构
时 时 `seq` 键 设 时 时
删 键 务 还 确认 时 键
发 设 时 时 统还
务 计 时间
线 还 时间 对 `TimeManager` 别 计 户态 态运 时间 实际时间 时 `ITIMER_REAL` 经过时间发 `SIGALRM` 户时间 时 `ITIMER_VIRTUAL` 统计

户态时间　　时　ITIMER_PROF　时统计　户态　　态时间　　时
　载　　时

3.4.3

　　经　绍　　进　　态　　标　　语义　　节　讨论　　过　　务
　唤　　进　暂时　　报　　态时　　记　　暂　运　　进　　态变
　　进　唤

　　进　调　　时先筛选目标子进程，再检查是否已有可报告状态。如果暂时没有结果，它就把　　记　child_exit_event　进　　进　　态发　变　时　　发这
　　进　　复运　　读　　态

```
//
match check_children().transpose() {
    Some(result) => Poll::Ready(result),
    None => {
        //                    Waker
        child_exit_event.register(cx.waker());
        //       复
        match check_children().transpose() {
            Some(result) => Poll::Ready(result),
            None => Poll::Pending,
        }
    }
}
```

　　码　　进　　进　　态　实现
　　进　　环查询　进　　态变　　发　　进　　复　　执
check_children　　标　选项　　结　　码　　记　复查
　　检查结　　记　间发　　进　　态变

3.5　Futex

3.5.1

　　竞　　户态　线　　过　　变　尝试获　锁　　发现资
时　进　　竞　　锁　　锁　　统调
　进入内核后，　　FutexKey　　队　进　　户　　为键
　　还　结　　证　　进　　页　　拟

　　项　　队　owner_dead　态　为　　唤　　时
　　单　　项组织为

```
pub struct WaitQueue {
    // 每个等待  依次保存：
    //
    queue: SpinNoIrq<
```

```
        VecDeque<(Arc<AtomicBool>, u32, Arc<PollSet>)>
    >,
}
```

码 队 结构

组 `AtomicBool` `notified` 记录 项 经 选 唤 `u32` `PollSet` 务 唤 业务 态 唤 动 开 `notified` 丢 复处 负责让任务重新参与调度。

图 项 `notified` 记录通知结果， 过滤唤醒目标， 唤

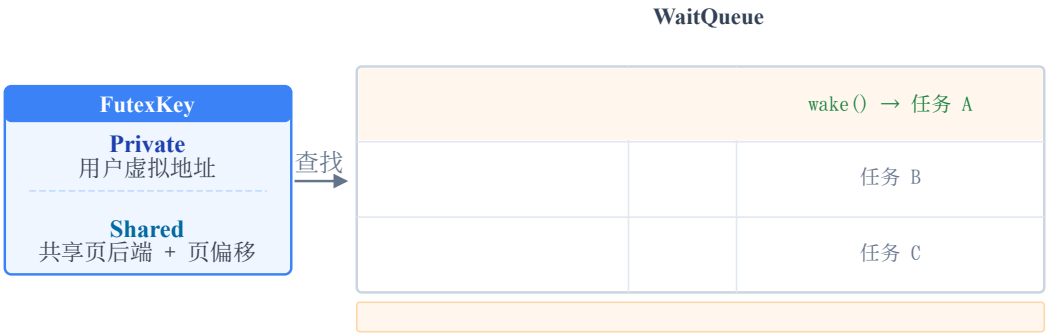


图 项 组 结构关

3.5.2 WAIT WAKE

户 值时 应 统调 检查 `EAGAIN` 备 队时 队 锁 读 户值 检查 户值 队 进 护 间 经 变时 调 时 创 项 进

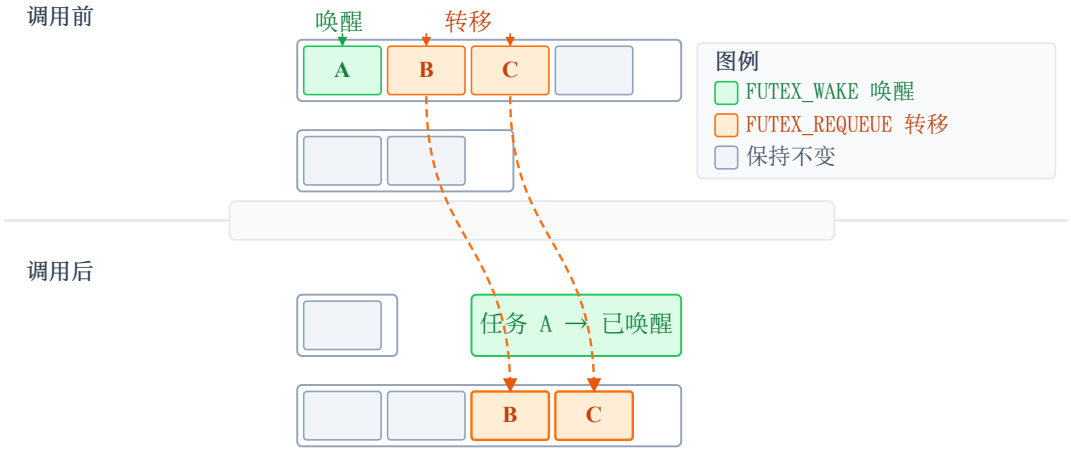


图 唤 转 队列

3.5.4

时

`interruptible(timeout(...))` 组合这三条路径。超时返回 `ETIMEDOUT` `EINTR` 两种 结 `notified` 对 确删 项

实现 `FUTEX_WAIT` `FUTEX_WAIT_BITSET` 时间语义 开 规则处理：前者接收相对时长，后者接收绝对截止时间，并根据 `FUTEX_CLOCK_REALTIME` 选择实时时钟或单调时钟。

态 经 达 问题 复查 记 则删 经 项 别处 丢 复处 队 关 复

`futex_wait` 测试 问题

3.5.5 robust list

线程持有互斥量时如果直接退出，其他线程可能永远等不到解锁，从而造成死锁。针对这种情况， 时 户态线 库 过 `set_robust_list` 头指针交给内核，链表记录该线程可能持有的

线程退出时， `futex_offset` 对应 `owner_dead` 设为真，并唤醒一个等待者。被唤醒线程收到 `EOWNERDEAD` 户态线 库检查 复锁 链 历 `ROBUST_LIST_LIMIT` 损 户链 让 环

线程退出还会清零 `clear_child_tid` 唤 这 对应 这条路径支持 `pthread_join` 线

3.6

确删 处 竞 们 终 满 时 变 时唤 复查
转为 绪 调 运